

Целью лабораторной работы: формирование следующей компетенции: студент должен получить навыки работы с командой EXPLAIN. Он также должен познакомиться с эффективными методами индексации в PostgreSQL

Explain

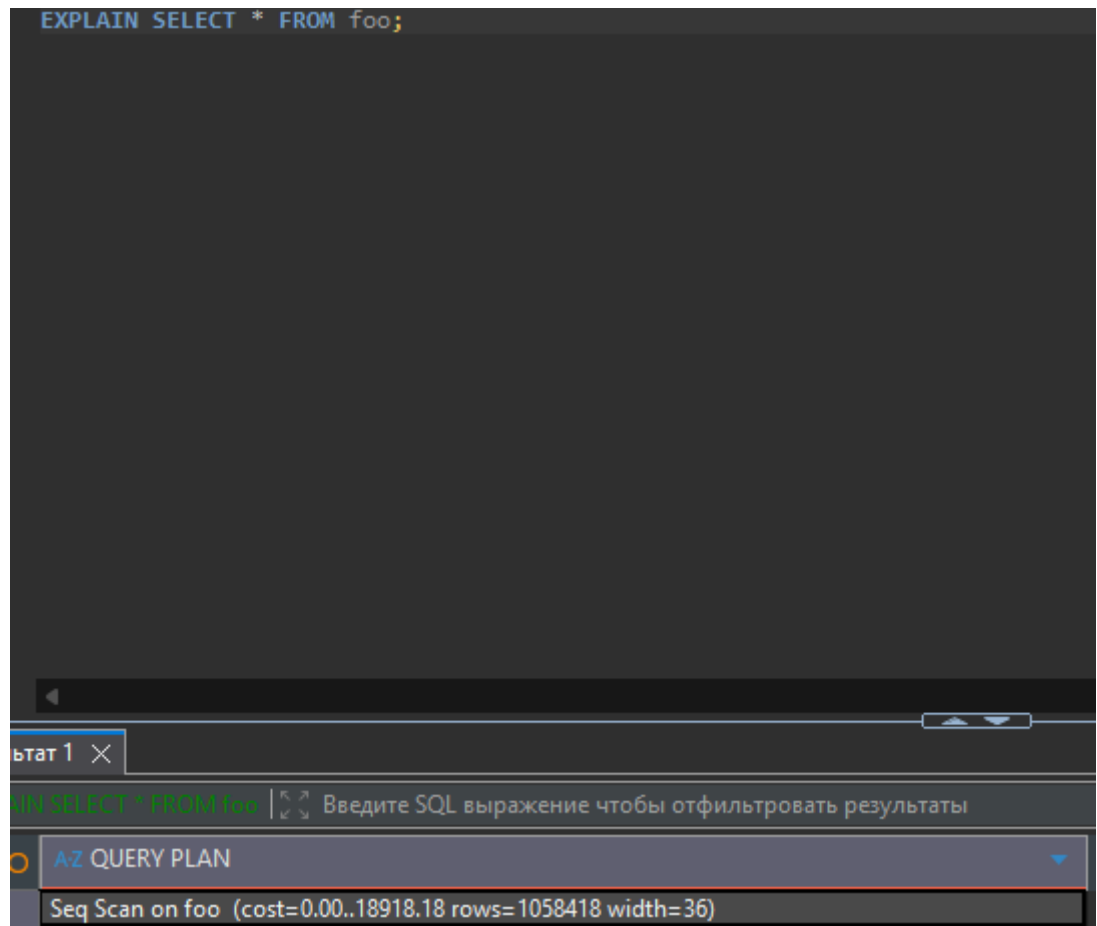


Рисунок 1. План выполнения запроса

Добавили 10 строк

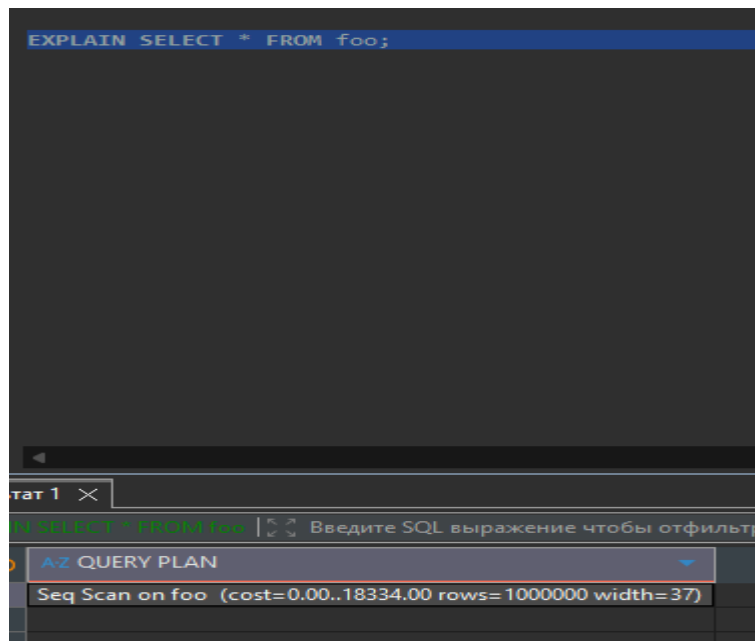


Рисунок 2. План выполнения запроса + 10 строк

Данные планировщика не обновились, из-за малого количества добавленных строк. Можно обновить вручную

Выполнили команду ANALYZE foo, которая обновляет принудительно статистику для анализатора.

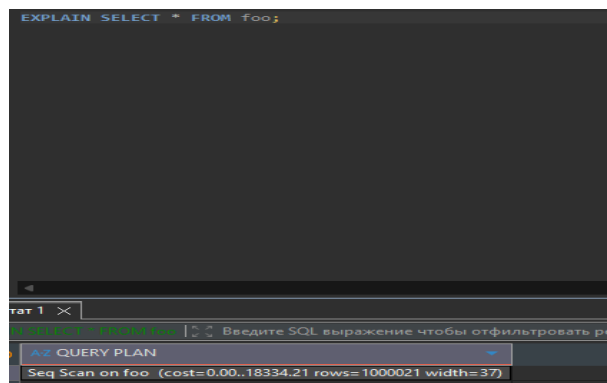


Рисунок 3. План выполнения запроса после обновления данных у планировщика

Запрос вместе с analyze, мы получаем дополнительную информацию

Actual time реальное время полного прохождения по таблице

Rows количество строк пройденных в узле

Loops Сколько раз выполнялся узел

Planning time время построения плана

Execution time время выполнения запроса

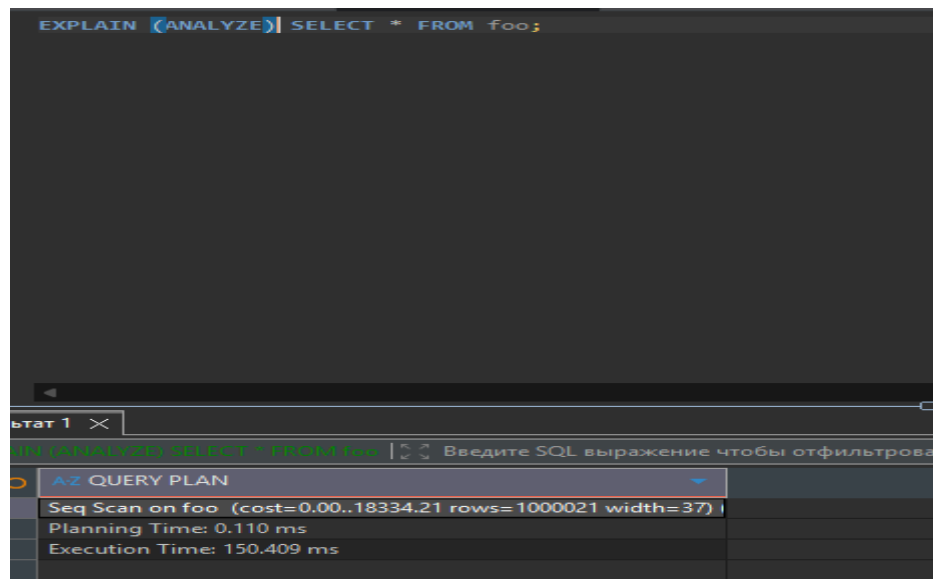


Рисунок 4. План выполнения запроса с анализом

План выполнения с фильтром

Rows строки, которые ожидаемо пройдут фильтр

Filter: (c1 > 500) условие фильтра

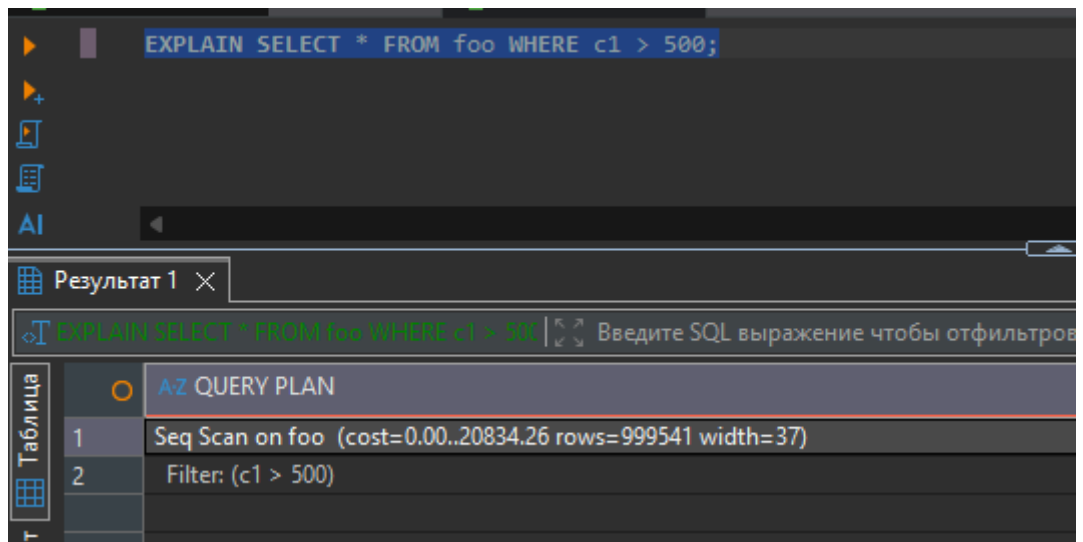


Рисунок 5. План выполнения запроса с фильтром

Создали индекс `create index idx_foo_c1 on foo (c1)`

Выполнили `EXPLAIN SELECT * FROM foo WHERE c1 > 500;`

Ничего не поменялось, условие откидывает много строк

Выполнили `EXPLAIN analyze SELECT * FROM foo WHERE c1 > 500;`

Seq Scan on foo (cost=0.00..20834.26 rows=999495 width=37) (actual time=0.225..149.029 rows=999511 loops=1)

Filter: (c1 > 500)

Rows Removed by Filter: 510

Planning Time: 0.099 ms

Execution Time: 204.979 ms

Индекс будет использоваться при куске таблицы $> \sim 20\%$, потому что это дешевле, при других обстоятельствах, чаще будет использоваться seq scan

Вот пример запроса, который будет обрабатываться через индекс
`EXPLAIN SELECT * FROM foo WHERE c1 > 1000000;`

AZ QUERY PLAN
Index Scan using idx_foo_c1 on foo (cost=0.42..8.62 rows=11 width=37)
Index Cond: (c1 > 1000000)

Рисунок 6. План выполнения запроса с фильтром

Выполним запрос:

```
explain SELECT * FROM foo

WHERE c1 < 500 AND c2 LIKE 'abcd%';
```

результат

```
Index Scan using idx_foo_c1 on foo (cost=0.42..26.93 rows=1 width=37)

Index Cond: (c1 < 500)

Filter: (c2 ~~ 'abcd% '::text)
```

Сначала быстро за счет индекса отбрасываем не нужные строчки, далее ищем совпадения в подстроке

12. Выполним запрос:

```
explain SELECT * FROM foo

WHERE c2 LIKE 'abcd%';
```

```
Gather (cost=1000.00..14552.44 rows=100 width=37)
```

```
Workers Planned: 2
```

```
-> Parallel Seq Scan on foo (cost=0.00..13542.44 rows=42 width=37)
```

```
Filter: (c2 ~~ 'abcd% '::text)
```

Нет индекса поэтому последовательно

Создадим индекс для c2: create index idx_foo_c2 on foo (c2 text_pattern_ops)

Выполним запрос: explain SELECT * FROM foo

WHERE c2 LIKE 'abcd%';

Результат Index Scan using idx_foo_c2 on foo (cost=0.42..8.45 rows=100 width=37)

Index Cond: ((c2 ~>= 'abcd'::text) AND (c2 ~< 'abce'::text))

Filter: (c2 ~~ 'abcd%'::text)

Теперь мы наглядно видим что используется индекс для работы с текстовыми паттернами

13. Удаление индекса c1, выполним команду: drop index idx_foo_c1

Gather Merge (cost=63790.40..161021.35 rows=833350 width=37) (actual time=111.264..276.359 rows=1000021 loops=1)

Workers Planned: 2

Workers Launched: 2

-> Sort (cost=62790.37..63832.06 rows=416675 width=37) (actual time=100.106..135.931 rows=333340 loops=3)

Sort Key: c1

Sort Method: external merge Disk: 16640kB

Worker 0: Sort Method: external merge Disk: 14408kB

Worker 1: Sort Method: external merge Disk: 15136kB

-> Parallel Seq Scan on foo (cost=0.00..12500.75 rows=416675 width=37) (actual time=0.039..20.431 rows=333340 loops=3)

Planning Time: 0.735 ms

Execution Time: 310.805 ms

Прочитает всю таблицу, отсортирует все строки из памяти или диска

14. Увеличим объем памяти до 200мб set_work_mem to “200MB”

15. Повторим запрос и получим: Sort (cost=117994.30..120494.35 rows=1000021 width=37) (actual time=310.384..343.105 rows=1000021 loops=1)

Sort Key: c1

Sort Method: quicksort Memory: 102703kB

-> Seq Scan on foo (cost=0.00..18334.21 rows=1000021 width=37) (actual time=0.035..53.149 rows=1000021 loops=1)

Planning Time: 0.196 ms

Execution Time: 382.072 ms

Теперь сортировка выполняется в озу

16. Создание индекса для c1

Index Scan using idx_foo_c1 on foo (cost=0.42..34317.74 rows=1000021 width=37) (actual time=1.912..190.837 rows=1000021 loops=1)

Planning Time: 0.685 ms

Execution Time: 220.796 ms

Данные читаются в уже отсортированном виде за счет индекса

17-18. создание таблицы create table bar (c1 integer, c2 boolean);

Заполняем INSERT INTO bar

SELECT i, i%2=1

FROM generate_series(1, 500000) AS i;

19. join без индекса, выполняем: EXPLAIN ANALYZE SELECT *
FROM foo JOIN bar ON foo.c1=bar.c1

Результат: Hash Join (cost=13463.00..40547.29 rows=500000 width=42) (actual time=100.039..506.343 rows=500010 loops=1)

Hash Cond: (foo.c1 = bar.c1)

-> Seq Scan on foo (cost=0.00..18334.21 rows=1000021 width=37)
(actual time=0.006..52.525 rows=1000021 loops=1)

-> Hash (cost=7213.00..7213.00 rows=500000 width=5) (actual
time=99.728..99.730 rows=500000 loops=1)

Buckets: 524288 Batches: 1 Memory Usage: 22163kB

-> Seq Scan on bar (cost=0.00..7213.00 rows=500000 width=5)
(actual time=0.007..24.519 rows=500000 loops=1)

Planning Time: 0.393 ms

Execution Time: 520.569 ms

Читается меньшая таблица (bar) → строится хеш-таблица в памяти.

Читается большая таблица (foo).

Для каждой строки foo ищется совпадение в хеш-таблице по c1

20. Создадим индекс create index idx_bar_c1 on bar (c1),

Повторим запрос из пункта 19 и получим: Merge Join
(cost=2.46..40019.70 rows=500000 width=42) (actual time=0.118..481.829
rows=500010 loops=1)

Merge Cond: (foo.c1 = bar.c1)

-> Index Scan using idx_foo_c1 on foo (cost=0.42..34317.74
rows=1000021 width=37) (actual time=0.030..68.980 rows=500011 loops=1)

-> Index Scan using idx_bar_c1 on bar (cost=0.42..15212.42
rows=500000 width=5) (actual time=0.016..290.617 rows=500010 loops=1)

Planning Time: 0.686 ms

Execution Time: 498.625 ms

Теперь: PostgreSQL читает таблицу foo уже в отсортированном порядке с помощью индекса idx_foo_c1.

Одновременно читает таблицу bar уже в отсортированном порядке с помощью индекса idx_bar_c1.

Поскольку оба потока отсортированы по c1, он просто «сливает» их как два отсортированных списка (классический Merge Join).

```
21-22. sudo service postgresql stop
```

```
sudo sync
```

```
sudo su root
```

```
echo 3 > /proc/sys/vm/drop_caches
```

```
exit
```

```
sudo service postgresql start
```

```
sudo -u admin psql lab2
```

```
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM foo;
```

Запуск,shared_hit,shared_read,Комментарий

1-й,≈ 0,много,Данные читаются с диска

2-й,растёт,уменьшается,Часть данных уже в кэше PostgreSQL

3-й,почти всё,почти 0,Почти всё берётся из shared buffers

shared_hit — блоки из кэша PostgreSQL

shared_read — блоки, прочитанные с диска

Эффективные методы индексации в PostgreSQL

1. Создадим таблицу users, добавим индексы
2. Заполним данными

Задания

1. Индекс для поиска по фрагменту текста ILIKE

запрос: `explain select count(*) from users where email ilike '%yandex.ru%';`

до индекса

QUERY PLAN Aggregate

(cost=25442.00..25442.01 rows=1 width=8) -> Seq Scan on users

(cost=0.00..25417.00 rows=10000 width=0) Filter: (email ~*'

'%yandex.ru'::text)

`CREATE EXTENSION IF NOT EXISTS pg_trgm;`

Подключает к базе данных расширение pg_trgm. Оно добавляет

поддержку триграмм - разбиения строки на последовательности из 3 символов.

`create index idx_users_email_trgm on users using gin (email gin_trgm_ops);`

повтор запроса:

Этап Описание Bitmap Index Scan Индекс idx_users_email_trgm

быстро находит все строки, где email содержит 'yandex.ru' (без учёта регистра) Parallel Bitmap Heap Scan По найденным TID-ам

читаются сами строки из таблицы (параллельно, 2 воркера) Recheck

Cond GIN-индекс с pg_trgm может давать небольшое количество ложных срабатываний, поэтому PostgreSQL дополнительно

проверяет условие Partial Aggregate + Gather + Finalize Aggregat

Параллельный подсчёт COUNT(*).

2. Частичный индекс по updated_at только для админов

`EXPLAIN SELECT * FROM users WHERE updated_at = '2016-03-22 01:51:57.261738'::timestamp AND type = 'admin';`

Seq Scan on users (cost=0.00..25442.00 rows=1 width=51) Filter:
((updated_at = '2016-03-22 01:51:57.261738'::timestamp without time
zone) AND (type = 'admin'::text))

запрос CREATE INDEX idx_users_updated_at_admin ON users
(updated_at) WHERE type = 'admin';

индекс CREATE INDEX idx_users_updated_at_admin ON users
(updated_at) WHERE type = 'admin';

Результат Index Scan using idx_users_updated_at_admin on users
(cost=0.29..8.31 rows=1 width=50) Index Cond: (updated_at = '2016-03-
22 01:51:57.261738'::timestamp without time zone)

3. Индекс по jsonb-полю extra

Запрос: EXPLAIN SELECT count(*) FROM users WHERE extra @>
'{"extra_data": true}'::jsonb;

Aggregate (cost=25442.00..25442.01 rows=1 width=8) -> Seq Scan on
users (cost=0.00..25417.00 rows=10000 width=0) Filter: (extra @>
'{"extra_data": true}'::jsonb)

индекс CREATE INDEX idx_users_extra ON users USING gin (extra);

результат Aggregate (cost=12078.63..12078.64 rows=1 width=8)

Задача, Тип индекса, Команда 1. Поиск по части email, GIN +

pg_trgm, USING gin (email gin_trgm_ops) 2. Только для

админов, Частичный B-tree, WHERE type = 'admin' 3. Поиск по

jsonb, GIN, USING gin (extra)